

해제 책임의 이관

scope 기반 RAII로 Addressables 수명을 자동화하다

한 문장

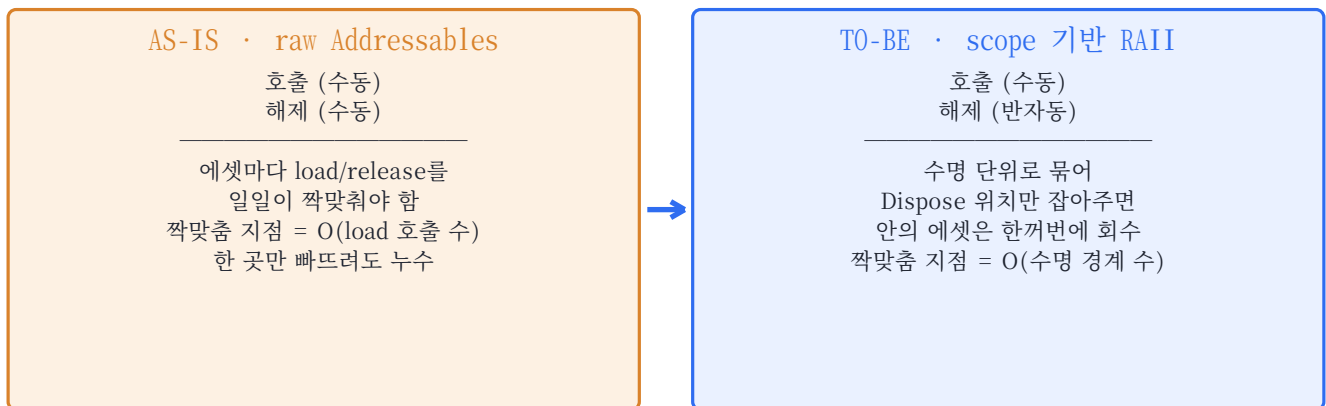
Addressables는 load↔release 짝맞춤을 "호출자"에게 위임한다.

나는 그 짝맞춤을 Scope 수명에 바인딩해 호출자에서 걷어냈다.

짝맞춤 지점을 $O(\text{load 호출 수}) \rightarrow O(\text{수명 경계 수})$ 로 압축.

원인과 결과 — 무엇이 바뀌었나

핵심 변화는 "해제를 없앴 것"이 아니라 "해제 책임을 호출자에서 수명 경계로 옮긴 것"이다.

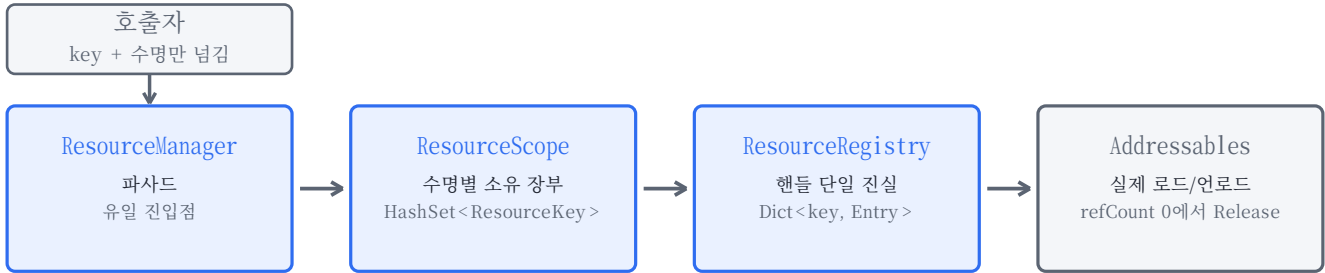


"반자동"의 정확한 의미 — 자동/수동이 섞인 게 아니라, 책임의 층이 나뉜 것

"이 수명이 언제 끝났나" 판단	도메인 로직 (스택 카운터 · 씰 전환 혹)	수동 — 경계를 코드로 선언
"그 안의 에셋을 실제로 놓기"	Scope.Dispose → Registry refCount 정산	자동 — 사람 관여 없음

구조 — Manager ↔ Scope ↔ Registry

외부는 오롯이 ResourceManager로만 요청한다. 호출자가 넘기는 건 "무엇을(key)"과 "어느 수명에(Scope)" 둘뿐.



설계 3원칙

① 핸들 소유는 레지스트리 한 곳

스코프는 티켓만 줘다 → "누가 Release하나" 문제 소멸

② 스코프 내 중복 로드 = refCount 1

HashSet.Add 반환값으로 첫 획득만 카운트. refCount = 소유한 서로 다른 스코프 수

③ 취소 ≠ 해제

토큰은 기다림 중단만. 수명은 스코프 소유 → use - after - release 구조적 불가

수명 계층 (2개 고정 → N개 enum 확장)



급소 — "그럼 Dispose는 누가 부르나?"

해제 책임은 사라진 게 아니라 호출자(수백 곳)에서 도메인 경계(몇 곳)로 이관됐다. 개수가 줄었을 뿐 규칙이 증발한 건 아니다 — 이 정직함이 오히려 방어적이다.

사례 · Popup 수명 = 팝업 스택의 0↔1 전이



안전망 (2차 경로) — 팝업을 열어둔 채 씬을 떠나면?

씬 전환 시퀀스가 `UI.Clear()` → `CloseAllPopupUI()` → (스택 강제 비움) → `DisposeScope(Popup)` 에 도달.

⇒ Popup 스코프는 두 경계에서 회수된다: (1) 정상 close, (2) 씬 이탈. 어느 쪽으로 나가도 새지 않는다.

검증 — "좋아졌다"가 아니라 숫자로

누수 계측 `AssertSceneHandlesCleared` — 씬 회전 직후 이전 핸들 전부 반납 검사 → 누수 0

재로드 churn `refCount` 공유로 왕복 시 재로드 flat

파생물 수명 아틀라스 클론 스프라이트까지 `OnReleased` 혹은 함께 파기

취소 정산 `RollbackLoad` — 취소/실패 시 방금 더한 `ref`만 되돌려 유령 참조 방지

포트폴리오 포지셔닝 (객관적 판단)

배치 — 유니티 헤드라인 서열

#1 AbilitySystem

게임플레이 · 헤드라인 (실제 동작 데모 필수)

#2 리소스 시스템

엔지니어링 성숙도 · 뒷받침 챗터

역할

대표작이 아니라 성숙도 뒷받침 챗터. "기존 코드를 안 망가뜨릴 사람"의 증거.

명명 주의

문서엔 scope - based RAII / scoped ownership로 정식 명명. "발명"처럼 쓰면 감점, "없었다"면 감점.

한 줄 축약 금지

load/release 짝맞춤 관찰 자체는 새 정보 아님. 가치는 실행+계측+앤티+절제. 한 줄은 TL;DR 입구로만.

레벨

강한 주니어 ~ 미들 초입 신호. 시니어 아키텍처로 과대표장 금지.

방어 질문 3종

Q. Addressables가 이미 refCount인데 왜 또?

Addressables refCount는 핸들 단위 수동 짝맞춤 — 호출부가 Release를 "기억"해야 하고 그 실패가 누수. 스코프는 기억을 구조로 대체.

Q. 본인이 만든 문제(2버킷)를 본인이 고친 것 아닌가?

맞다, 숨기지 않는다. 계측으로 한계 발견 → 개인 주의력이 아니라 구조로 재발 차단. 레거시 개선은 실무의 일상.

Q. 1인 프로젝트에 오버엔지니어링 아닌가?

측정치가 필요성 증명. 반대로 DDOL 대공사 · 스트리밍은 측정 근거로 제외 = 필요한 것만 했다는 증거.